ColliderAgent

安装、测试与运行注意事项

从零依赖安装到端到端冒烟验证的完整操作手册

适用对象：在本机（Ubuntu 24.04，无 `sudo`、无 Docker、24 核、Python 3.12）上安装并使用 **ColliderAgent** 的研究人员与工程使用者。文档覆盖三条主线：**(i)** 依赖安装与环境配置；**(ii)** 测试与验证（冒烟测试、单元/集成测试、端到端核对）；**(iii)** 运行注意事项与故障排查（云/本地双模式、机器热节流、物理口径坑）。

验证环境	本工作区实机（见第 4 节）
验证日期	2026 年 9 月（本文档写作时实跑核对）
测试结果	`pytest tests/` → **38 passed**； DeepSeek 运行时工具循环冒烟测试通过
对应代码	`ColliderAgent/`（含 `python-agent/`、`deepseek-agent/`）

本文档所有路径以工作区根目录*/media/ubuntu/6156e08b-fdb1-4cde-964e-431f74a6078e/Program/ColliderSim/*

为基准。

目录

1 文档目的与适用对象 **4**
1.1 ColliderAgent 是什么 . . . 4
1.2 本文档覆盖什么 . . . 4

2 系统要求与总体结构 **4**
2.1 硬件与系统要求 . . . 4
2.2 仓库结构（本工作区实际布局） . . . 5
2.3 三条执行路径：如何选 . . . 5

3 安装依赖 **6**
3.1 安装流程图 . . . 6
3.2 系统依赖 . . . 6
3.3 Python 依赖安装 . . . 7
3.4 配置 `.env` . . . 8
3.5 外部 HEP 工具链 . . . 8
3.6 一键安装 DeepSeek 运行时 . . . 9
3.7 本机对上游仓库的修复（务必保留） . . . 10

4 安装验证 **10**
4.1 本机实测版本矩阵 . . . 10
4.2 逐条验证命令 . . . 11
4.3 本机实测验证输出 . . . 12

5 测试 **12**
5.1 测试分层 . . . 12
5.2 L0 / L2：Python 侧测试 . . . 13
5.3 L1：DeepSeek 运行时冒烟测试 . . . 13
5.4 L3：端到端（本地模式） . . . 14
5.5 物理结果核对清单 . . . 15

6 运行注意事项 **15**
6.1 每次运行前的固定动作 . . . 15
6.2 云模式 vs 本地回退 . . . 16

6.3 机器散热与资源节流（本机重点） 16
6.4 运行目录与产物 17
6.5 修改 agent 内部文件的正确姿势 17
6.6 常见故障快速排查 18
6.7 安全与保密 18

A 命令速查表 **19**

B 环境变量速查 **19**

C 相关文件与文档索引 **20**

D 参考链接 **21**

1 文档目的与适用对象

1.1 ColliderAgent 是什么

ColliderAgent 是一套**端到端对撞机唯象自动化架构**：把一个 LaTeX 拉氏量变成可发表的物理图。它由三部分组成：

- **子代理定义**（`src/agents/`）：`model-generator`（LaTeX → FeynRules → UFO）、`collider-simulator`（MadGraph5 + Pythia8/Delphes 事件产生）、`event-analyzer`（MadAnalysis5 切流/直方图）、`pheno-analyzer`（全流程编排）。
- **技能模块**（`src/skills/`）：13 个技能，覆盖 FeynRules 模型生成/校验、UFO 导出、MG5 模拟、MA5 分析、micrOMEGAs、CalcHEP、Magnus 云接口、轻标量衰变宽度等。
- **程序化入口**：`python-agent/`（Google ADK 智能体 + `CLI.py` 交互配置）与 `scripts/`（Magnus 蓝图入口脚本）；本工作区另有 `deepseek-agent/`——把 `collider-simulator` 子代理的“大脑”换成 **DeepSeek API** 的独立运行时（`agent.py`），无需 Claude Code。

标准流水线为：

LaTeX 拉氏量 → FeynRules `.fr` → UFO 模型 → MG5 事例 → Pythia8 簇射 → Delphes 探测 → MA5 分析 → 图 / 排除线

1.2 本文档覆盖什么

章节	内容
第 2 节	系统要求、仓库结构、三条执行路径的选型建议
第 3 节	依赖清单、一键安装脚本、手工安装步骤、外部 HEP 工具链
第 4 节	安装验证（本机实跑结果与逐条核对命令）
第 5 节	测试体系：冒烟测试、pytest 套件、端到端验证与核对清单
第 6 节	运行注意事项：云/本地模式、热节流、物理口径、故障排查
附录	命令速查表、环境变量清单、文件与文档索引

阅读前提

本文档描述的是**本工作区已落地的实机状态**（含若干对上游仓库 bug 的本地修复，见第 3.7 节）。若你在别处重新克隆官方仓库，第 3.7 节列出的修复仍需重新施加，否则会出现“导入即失败”。

2 系统要求与总体结构

2.1 硬件与系统要求

项目	实测/建议值
操作系统	Ubuntu 24.04 LTS（其他 Linux 亦可）
CPU	建议 ≥ 8 核；探测/事例产生强烈并行（本机 24 核）
内存	≥ 16 GB（MA5 + ROOT + 多进程 Wolfram 内核吃内存）
磁盘	≥ 20 GB 空闲（MG5 + LHAPDF + Delphes + micrOMEGAs + 事例样本）
Python	**3.11+**（`pyproject.toml` 要求；本机 3.12.3）
编译器	gcc/g++ ≥ 9（本机 13.3.0，用于 MA5/Delphes/micrOMEGAs 编译）
权限	普通用户即可（**无需 sudo**）；本地 Magnus 部署才需要 Docker
散热	**本机为重点约束**：idle 即 67–71 °C，满载可冲 100 °C crit，必须节流（见第 6.3 节）

2.2 仓库结构（本工作区实际布局）

```
ColliderSim/
+-- ColliderAgent/                    # 主仓库
|   +-- README.md / README_zh.md      # 上游说明（中/英）
|   +-- DEPENDENCIES.md               # 本机依赖完成情况记录
|   +-- local_env.sh                  # 环境变量一键 source（本工作区新增）
|   +-- src/agents/                   # 4 个子代理定义
|   +-- src/skills/                   # 13 个技能模块
|   +-- python-agent/                 # ADK 智能体（含 .venv、tests/）
|   |   +-- agent.py / CLI.py
|   |   +-- tools/ utils/ prompts/ templates/
|   |   +-- tests/                    # pytest 套件（38 用例）
|   +-- deepseek-agent/               # DeepSeek 运行时（本工作区新增）
|   |   +-- agent.py / .env / ref/ / scripts/ / src/
|   +-- scripts/                      # Magnus 蓝图入口脚本
|   +-- paper-reproduction/           # 8 篇论文的复现提示词
+-- setup_collider_agent.sh           # DeepSeek 代理一键安装脚本
+-- MadGraph5/MG5_aMC_v3_7_2/         # 外部工具链
+-- MadAnalysis5/madanalysis5-1.11.1/
+-- delphes/  micromegas_6.3.0/  Models/  darkhiggs/
+-- docs/                            # 本工作区文档（含本文）
```

2.3 三条执行路径：如何选

路径	入口	适用场景
A. Claude Code 技能/子代理	`src/agents/` + `src/skills/` 拷到`~/.claude/`	官方推荐路径；完整子代理 + 技能支持。需要 Claude Code，且重活走 Magnus 云。
B. Python ADK 智能体	`python-agent/agent.py`（或 `CLI.py` 交互配置）	程序化调用、脚本化、自定义工作流；不依赖任何 AI 编码助手。工具调用走 Magnus 云蓝图。
C. DeepSeek 运行时	`deepseek-agent/agent.py`	**本工作区实际使用的路径**：模型换成 `deepseek-v4-flash`，本地工具循环（bash/read/write/edit/glob/grep）；Magnus 不可用时可回退本地 MG5/MA5。

选型建议

若只是要跑通流水线并产出物理结果，用路径 C（`deepseek-agent/`）最省事；若要复用官方全部 13 个技能与 4 个子代理，用路径 A；若要把流水线嵌进自己的 Python 程序，用路径 B。

3 安装依赖

3.1 安装流程图

```
(1) 系统包              -> python3-venv / gcc-g++ / make / cmake / git / zlib 等
(2) Python 虚拟环境     -> python-agent/.venv (requirements.txt / uv sync)
(3) Python 依赖         -> google-adk, litellm, sympy, pyyaml, wolframclient, ...
(4) .env 配置           -> LLM key + MG5/MA5/Delphes/micrOMEGAs 路径
(5) 外部 HEP 工具链     -> MG5(+LHAPDF) / MadAnalysis5 / Delphes / Pythia8 / micrOMEGAs
(6) Wolfram 栈          -> Wolfram Engine + FeynRules (+ FeynCalc)
(7) DeepSeek 代理       -> ./setup_collider_agent.sh
(8) 验证                -> source local_env.sh && pytest tests/ && 冒烟测试
```

3.2 系统依赖

```
# 需要 root 的仅此一步；若已有这些包可跳过
sudo apt-get update
sudo apt-get install -y build-essential gfortran cmake git \
    python3-venv python3-dev zlib1g-dev libboost-all-dev \
    libgsl-dev libssl-dev
```

无 sudo 环境怎么办

本机为普通用户（无 sudo、无 Docker）。全部 Python 依赖与外部 HEP 工具链（MG5、MA5、Delphes、LHAPDF、micrOMEGAs）均可下载源码后**在用户目录内源码编译**，已验证可行。唯一无法绕开 root 的是系统包与 CPU 频率限制（`cpupower frequency-set` 需要 sudo，见第 6.3 节）。

3.3 Python 依赖安装

方式一：uv（官方推荐）

```
cd ColliderAgent/python-agent
curl -LsSf https://astral.sh/uv/install.sh | sh   # 若未安装 uv
uv sync                                            # 按 pyproject.toml 建环境
```

方式二：pip + venv（本机实际采用）

```
cd ColliderAgent/python-agent
python3 -m venv .venv
./.venv/bin/pip install -U pip
./.venv/bin/pip install -r requirements.txt
# 本机额外补装（仓库 requirements 未含但代码需要）：
./.venv/bin/pip install authlib magnus-sdk pytest
```

镜像源坑（本机实测）

配置的**清华 PyPI 镜像在本网络返回 403**，必须改用可用镜像，例如：

```
./.venv/bin/pip install -r requirements.txt \
  -i https://mirrors.aliyun.com/pypi/simple/
```

`pyproject.toml` 的 `[tool.uv.sources]` 注释里默认写的也是清华源，若 `uv sync` 卡住/报 403，请改源或改用 pip。

Python 依赖清单（`requirements.txt` 要点）

包	用途 / 备注
`google-adk`	Google Agent Development Kit —— ADK 智能体框架（本机 2.8.0）
`litellm>=1.0.0`	统一 LLM 接口（本机 1.98.0）
`sympy` / `pyparsing`	符号计算与解析
`requests` / `beautifulsoup4` / `lxml`	参考模型下载与网页解析
`python-dotenv` / `pyyaml`	`.env` 读取、YAML 仿真配置
`wolframclient`	与 Wolfram 内核通信（FeynRules 校验/UFO 导出）

包	用途 / 备注
pexpect	MadGraph5 交互式会话管理（**必需**，缺了 MG5 工具全废）
authlib	magnus-sdk 的加密依赖（**requirements 未列，需手装**）
magnus-sdk	Magnus 云平台 SDK（本机 0.8.2；官方 README 要求 ≥0.7.0）
pytest	测试框架（本机 9.1.1）

3.4 配置 .env

```
cd ColliderAgent/python-agent
cp .env.example .env
# 然后编辑 .env，填入真实值（见下表）
```

变量	必需	说明 / 本机取值
COLLIDER_AGENT_MODEL	是	LiteLLM 模型串，如 openai/gpt-5-mini
OPENAI_API_KEY	是	LLM 提供方密钥（**切勿提交进 git**）
OPENAI_BASE_URL	否	自定义端点（代理/兼容服务）
MG5_PATH	是 †	MG5 的 bin 目录
MA5_PATH	是 †	MadAnalysis5 的 bin 目录
DELPHES_PATH	否	不设则从 MG5_PATH 推导
MICROMEGAS_ROOT	否	micrOMEGAs 根目录
WOLFRAM_KERNEL_PATH	否	Wolfram 内核路径
WOLFRAMSCRIPT_PATH	否	wolframscript 路径
MAGNUS_ADDRESS / MAGNUS_TOKEN	是 ‡	走 Magnus 云蓝图时必需
DEFAULT_NEVENTS / DEFAULT_EBEAM1,2	否	仿真默认值（10000 / 6500+6500）

† 若走 Magnus 云端，本地路径可不设；本地回退模式必设。

‡ 仅当使用需要 Magnus 的工具（validate_feynrules、generate_ufo_model、madgraph_compile/launch）时必需；用 scripts/ 本地入口可不设。

3.5 外部 HEP 工具链

这一层是**最容易失败**的部分。上游 README 假定这些工具通过 Magnus 云提供，因此本地安装属于“自建”，需要自己解决编译与依赖。

工具	本机位置	安装要点
MadGraph5_aMC	`MadGraph5/MG5_aMC_v3_7_2/` (v3.7.2)	官方 tarball（launchpad），纯 Python 无需编译。
LHAPDF 6.5.6	`MG5_aMC_v3_7_2/HEPTools/lhapdf6_py3/`	MG5 内建安装；PDF 网格 `cteq6l1`（lhaid 10042）。
Pythia8	`/usr/local/pythia8`	源码 `./configure && make`，由 agent 环境检查自动发现。
Delphes 3.5.x	`delphes/`（已编译）	需先装 ROOT；编译后在 `input/mg5_configuration.txt` 注册 `delphes_path`。
MadAnalysis5 1.11.1	`MadAnalysis5/madanalysis5-1.11.1/`	独立 venv `ma5_venv/`（`requirements.txt`，同样需换镜像源）。
micrOMEGAs 6.3.0	`micromegas_6.3.0/`	Zenodo tarball；**CalcHEP 必须单线程 make**（并行会损坏 `lib/serv.a`）。
Wolfram Engine 14.3	`/usr/local/Wolfram/...`	免费 license；在**容器内**激活（license 绑定容器机器标识）。
FeynRules 2.3.49	`~/.Wolfram/Applications/FeynRules`	必须**自定位补丁**（见第 3.7 节）。
FeynCalc 10.1.0	`~/.Wolfram/Applications/FeynCalc`	本仓库流程不直接调用，但宽度核验可用。

LHAPDF 的 LD_LIBRARY_PATH 必须在进程启动前设好

MG5 的 LHAPDF Python 绑定在进程启动早期就被 glibc 缓存动态库路径，**运行中途再 export 无效**。本工作区提供两处保障：

- `ColliderAgent/local_env.sh` —— 先 source 再跑；
- `MadGraph5/.../run_mg5.sh` —— 直接封装好 LD_LIBRARY_PATH。

3.6 一键安装 DeepSeek 运行时

```
cd /media/ubuntu/.../Program/ColliderSim
./setup_collider_agent.sh              # 安装到 ColliderAgent/deepseek-agent
./setup_collider_agent.sh --force      # 重新拷贝并刷新 .env
DEST=/tmp/agent-test ./setup_collider_agent.sh   # 装到别处
```

脚本是**幂等的**：重复运行安全，除非加 `--force`，否则不会覆盖已有 `.env`。可覆盖的环境变量：`DEEPSEEK_API_KEY`、`DEEPSEEK_MODEL`、`DEEPSEEK_BASE_URL`、`DEST`、`VENV_DIR`。

修改仓库文件后必须重跑安装脚本

`setup_collider_agent.sh` 把 `agent.py`、技能文件、蓝图脚本**内嵌**在脚本里再写到目标目录。你如果直接改了 `ColliderAgent/src/skills/...` 或 `scripts/run_madgraph_*.py`，必须重跑该脚本，否则目标目录里仍是旧副本。

3.7 本机对上游仓库的修复（务必保留）

以下 4 类问题在本机已修复，否则**安装即失败**（无法 import 或跑不通）：

问题	本地修复
`utils/install_madgraph.py`、`install_pythia.py`、`install_delphes.py` **仓库中缺失**，却被 `utils/madgraph_setup.py` 导入	重建为“探测优先”的安装器（env/dotenv 感知）；`check_madgraph_environment()` 现能识别 MG5、Pythia8、Delphes 及 3 张探测器卡。
`tools/madgraph_tools.py` 缺少 `connect_to_madgraph`	以 `connect_to_madanalysis` 的旧别名重新导出。
`tools/simulation_yaml_to_madgraph.py` 缺 3 个内部函数；`_resolve_model_path` 返回类型错误	补 `_build_setup_script`/`_build_launch_script`/`_get_next_run_number`；让路径返回普通字符串；补 `import re`。
`scripts/run_madgraph_compile.py`、`run_madgraph_launch.py`、`run_madanalysis_process.py` **硬编码容器路径**	可执行文件查找改为尊重 `MG5_PATH` / `MA5_PATH`，从而能纯本地运行。
FeynRules 的 `<< FeynRules` 找不到 `$FeynRulesPath`	给 `FeynRules.m` 打了**自定位回退补丁**，使裸加载可用（ColliderAgent 模板从不自行设置该变量）。
MA5 启动时更新检查器只捕获 `ConnectionError`，本网络下 GitHub API 超时（`ReadTimeout`）导致启动崩溃	`madanalysis/system/checkup.py` 捕获异常列表加入 `requests.exceptions.Timeout`。

4 安装验证

4.1 本机实测版本矩阵

组件	路径 / 标识	状态
Python	3.12.3（`python-agent/.venv` 同版本）	通过
gcc / g++	13.3.0	通过
CPU	24 核	通过
MadGraph5_aMC	v3.7.2，`bin/mg5_aMC`	通过

组件	路径 / 标识	状态
LHAPDF	6.5.6（cteq6l1），含 systematics	通过
Pythia8	/usr/local/pythia8/bin/pythia8-config	通过
Delphes	delphes/DelphesHepMC2 + CMS/ATLAS/ILD 卡	通过
MadAnalysis5	v1.11.1，bin/ma5 + ma5_venv	通过
micrOMEGAs	6.3.0，MSSM/main 已编译	通过
Wolfram Engine	14.3（WolframKernel + wolframscript）	通过
FeynRules / FeynCalc	2.3.49 / 10.1.0，~/.Wolfram/Applications	通过
google-adk / litellm	2.8.0 / 1.98.0	通过
magnus-sdk	0.8.2	通过
wolframclient	1.4.0（1+2→3 已验证）	通过

4.2 逐条验证命令

```
# --- 0) 载入环境 (每次新 shell 都要做)---
cd /media/ubuntu/.../Program/ColliderSim/ColliderAgent
source local_env.sh

# --- 1) Python 侧依赖是否齐全 ---
cd python-agent
./.venv/bin/python -c "import agent, tools, utils; print('imports OK')"
./.venv/bin/python -m pip list | grep -Ei 'google-adk|litellm|magnus|wolframclient|pytest'

# --- 2) 外部工具可达性 ---
"$MG5_PATH/mg5_aMC" --version          # 期望 3.7.2
"$MA5_PATH/ma5" --help >/dev/null && echo "ma5 OK"
ls "$DELPHES_PATH/DelphesHepMC2"       # 期望存在
/usr/local/pythia8/bin/pythia8-config --version
ls "$MICROMEGAS_ROOT/MSSM/main"        # 期望存在

# --- 3) Wolfram 栈 ---
wolframscript -code "1+1"              # 期望 2
wolframscript -code '<<FeynRules; $FeynRulesPath'   # 期望打印路径

# --- 4) DeepSeek 运行时冒烟 ---
cd ../deepseek-agent
../python-agent/.venv/bin/python -c "import agent; agent.build_system_prompt()" \
  | head -5
```

4.3 本机实测验证输出

实测记录（本次文档写作时复跑）

- `import agent, tools, utils` → **imports OK**（全部工具模块可导入）。
- `pytest tests/ -q` → **38 passed, 3 warnings**，耗时 13.5 s（3 条 warning 为 `tests/test_madgraph_tools.py` 中测试函数 `return bool` 而非 `assert` 引发的 `PytestReturnNotNoneWarning`，不影响通过）。
- DeepSeek 运行时冒烟：`agent.py "Run the bash tool…pwd"` → 2 轮迭代，模型正确调用 `bash` 并返回工作目录，**通过**。
- 编码依赖：`wolframclient` 桥接评估 `1+2 = 3`，**通过**。
- MA5 本地 hook `connect_to_madanalysis` → **SUCCESS, RC=0**。
- `local_env.sh` 已写入本机绝对路径；`python-agent/.env` 内 `MG5_PATH/MA5_PATH/DELPHES_PATH/MICROMEGAS_ROOT/ WOLFRAM_*` 均指向真实路径。

两个未完成的验证项（已知、可接受）

1. **UFO 全量导出未跑完**：FeynRules `WriteUFO` 会拉起 16 个并行 Wolfram 内核，本机散热不足**曾触发过热关机**。已验证到"FeynRules 模型处理阶段"，全量 `WriteUFO` 建议节流后择机补跑（方法见第 6.3 节）。
2. **Magnus 云连通性**：本机 `MAGNUS_ADDRESS` 指向 `http://162.105.131.134:3011`，实测 **connection refused**；因此凡走 Magnus 的工具需回退到本地脚本（见第 6.2 节）。

5 测试

5.1 测试分层

层级	命令 / 位置	验证目标
L0 导入冒烟	`python -c "import agent, tools"`	依赖齐全、无语法/缺失模块错误。这是**最先该跑**的一步。
L1 运行时冒烟	`deepseek-agent/agent.py "run pwd…"`	LLM API 可达、工具循环（tool-calling）能闭合、`.env` 生效。
L2 单元/集成	`pytest tests/`	工具函数逻辑、仿真工作流 YAML 生成、E2E 管线的非计算部分（38 用例）。
L3 端到端物理	`scripts/` 本地入口 + 已有事例样本	真实物理链路：FeynRules 校验 → UFO → MG5 → MA5。
L4 结果核对	第 5.5 节清单	数值锚点与参考（FORESEE / 文献 / 手算）逐条对齐。

5.2 L0 / L2：Python 侧测试

```
cd ColliderAgent/python-agent

# 只收集用例、不执行（检查测试文件本身是否有 import 错误）
./.venv/bin/python -m pytest tests/ --collect-only -q

# 全量跑
./.venv/bin/python -m pytest tests/ -q

# 只跑单个文件 / 用例
./.venv/bin/python -m pytest tests/test_e2e_pipeline.py -q
./.venv/bin/python -m pytest tests/test_simulation_workflow.py -q -k yaml
```

测试套件构成

tests/ 文件	覆盖内容
test_e2e_pipeline.py	端到端管线编排（非计算部分）
test_madgraph_tools.py	MG5 交互封装（含 pexpect 会话）
test_simulation_workflow.py	YAML 仿真配置生成与解析
tests/assets/	参考 .fr 模型与脚本（如 SM.fr）

本机结果：38 passed，3 warnings in 13.53s。历史上这里曾是**收集错误（collection errors）**，原因是第 3.7 节的缺失模块；修复后全绿。

pytest-timeout 未安装，别加 --timeout

本机 pytest 9.1.1 未安装 pytest-timeout 插件，--timeout=600 会被当作无法识别的参数而直接报错（error: unrecognized arguments: --timeout=600）。需要超时保护请用 shell 层：timeout 600 ./.venv/bin/python -m pytest tests/ -q。

5.3 L1：DeepSeek 运行时冒烟测试

```
cd ColliderAgent/deepseek-agent

# (a) 只拼装系统提示，不调用 API（零成本）
../python-agent/.venv/bin/python -c "import agent; agent.build_system_prompt()" | head -20

# (b) 完整工具循环冒烟（会消耗一次 API 调用）
../python-agent/.venv/bin/python agent.py \
  "Run the bash tool with the command 'pwd', then report what you see in one sentence."

# (c) 从提示词文件跑真实任务
../python-agent/.venv/bin/python agent.py --task-file prompt.md \
  --workdir /path/to/analysis \
  --progress-file progress/step2_madgraph.md
```

可选参数：--workdir（工具与产物的工作目录，默认当前目录）、--max-iterations（默认 40

轮）、--model（覆盖 DEEPSEEK_MODEL）、--progress-file。

实测（步骤 b）

输出形如：

```
[agent] model=deepseek-v4-flash base_url=https://api.deepseek.com
[agent] workdir=.../ColliderAgent/deepseek-agent max_iterations=40
[agent] --- iteration 1/40 ---
[tool] bash({"command": "pwd"})
[agent] --- iteration 2/40 ---
[agent] ==== FINAL ANSWER ====
The working directory is .../ColliderAgent/deepseek-agent
```

可见：模型正确选择工具、工具执行成功、循环在 2 轮内收敛。

5.4 L3：端到端（本地模式）

Magnus 不可用时，用 scripts/ 下的入口脚本在本地跑同一套逻辑（这些脚本与云端蓝图同源，仅路径查找改为尊重 MG5_PATH/MA5_PATH）：

```
cd ColliderAgent
source local_env.sh
export PY=python-agent/.venv/bin/python

# (1) FeynRules 模型校验（需要 Wolfram + FeynRules）
$PY scripts/run_feynrules_validation.py --model tests/assets/SM.fr --symbol SM

# (2) UFO 导出（重！散热不足时按第 9.2 节节流）
$PY scripts/run_ufo_generation.py --fr <model.fr> --lagrangian <SYMBOL> --target ./UFO

# (3) MadGraph5：先编译（列图、编译矩阵元），再生事例
$PY scripts/run_madgraph_compile.py --ufo ./UFO \
    --process "p p > mu+ mu-" --target ./pp_mumu
$PY scripts/run_madgraph_launch.py --process ./pp_mumu \
    --commands "done\nset nevents 1000\ndone"

# (4) MadAnalysis5 分析（直方图 / 切流）
$PY scripts/run_madanalysis_process.py \
    --events ./pp_mumu/Events/run_01/unweighted_events.lhe.gz \
    --script ./ma5_script.txt --level normal
```

MG5 的两步式工作流

MadGraph5 必须**先 compile 再 launch**：编译阶段枚举费曼图并生成/编译矩阵元，launch 阶段才真正产生事例。这样同一个已编译过程可以被多种 run 配置复用，省掉重复编译开销。

MG5 命令行的“两个 done”状态机

madgraph-launch 通过 --commands 与 MG5 交互，必须遵守状态机：**所有 set … 行都要写在第二个 done 之前**。写错顺序会导致参数被静默忽略（用默认值跑），表现为“改了 nevents 却还是默认事例数”。

5.5 物理结果核对清单

跑完 L3 不等于结果正确。以下清单来自本工作区多轮踩坑总结，**建议逐条打勾**：

	核对项
☐	$\sigma(b\bar{b})$ 与文献/独立值一致（13 TeV LO ~400–450 µb，NNLO ~500 µb）
☐	产生截面读取位置正确：LHE `<init>` 块的截面（XSECUP，单位 pb）在**第二行第一字段**，第一行是束流 ID（2212）。读错会错 $\sim 2\times10^5$ 倍
☐	单位换算不混：pb / µb / fb 相差 10^3、10^6；$N_{\rm sig}=\sigma[\rm fb]\times L[\rm fb^{-1}]\times\cdots$
☐	分支比公式与参考**逐项**一致（例如 $\mathrm{BR}(B^+\to K^+\phi)=5.6\sin^2\theta(1-m^2/m_B^2)^2$，$m_B=5.279$，**不是** $m_b=4.18$）
☐	衰变宽度口径正确：轻子道 Winkler 式；强子道 ChPT 迹反常形式因子**仅** $m<1.3$ GeV；粲道阈值用 $2m_D\approx3.73$ GeV（**不是** $2m_c=2.55$）
☐	$c\tau$ 手算抽查：$c\tau=\hbar c/(\sin^2\theta\,\Gamma_{\rm SM}(m))$ 与结果表一致
☐	长度单位一致（**mm vs m**）：宽度模块给 `ltime_mm`，而盒坐标是**米**，混用会少除 1000
☐	接受度量级合理（$\sim10^{-4}$–10^{-3}），确认“只对穿盒事件平均”的虚高 bug 未复现（该 bug 可虚高 ~250 倍；正确做法是对**所有**事件取平均，未命中记 0）
☐	ray–box 相交使用**单位方向余弦**：$t=\mathrm{box}/p_{\rm axis}$ 中的 $p_{\rm axis}$ 必须是方向余弦；直接传动量分量会差 $\sim10^5$ 量级
☐	**几何单点手算**：取一个事例手算 slab 入口（方向 x 分量 0.65 → $x\in[26,36]$ 入口 $\approx26/0.65\approx40$ m），与代码 `lmin` 比对
☐	**上下界都对照**参考：只有下边界（长寿命端）一致可能是单位 bug 巧合；上边界（大 $\sin\theta$、短寿命端）对 MC 谱尾部敏感，必须一起对
☐	绘图数据与原始 JSON 一致（抽查一个网格点，比较 `N_sig/excluded`）
☐	换探测器时口径全对齐：对撞类型（collider vs fixed_target）、BR 形式（foresee / winkler / inclusive，三者对 $B^+\to K^+\phi$ 可差 ~34 倍）、可见度（track vs all）

6 运行注意事项

6.1 每次运行前的固定动作

```
cd /media/ubuntu/.../Program/ColliderSim/ColliderAgent
source local_env.sh          # MG5/MA5/Delphes/LHAPDF/micrOMEGAs 路径 + LD_LIBRARY_PATH
# 然后才跑 agent.py 或 scripts/*.py
```

不 source 会怎样

`LD_LIBRARY_PATH` 必须在进程启动前设置（glibc 早期缓存），且 `scripts/` 入口靠 `MG5_PATH/MA5_PATH` 找可执行文件。新开终端忘了 source，最典型症状是 MG5 因找不到 LHAPDF 动态库而报错，或脚本提示找不到 `mg5_aMC`。

6.2 云模式 vs 本地回退

通道	行为
Magnus 云	`python-agent/tools/` 导出的 `validate_feynrules`、`generate_ufo_model`、`madgraph_compile`、`madgraph_launch` 走 `MAGNUS_ADDRESS`/`MAGNUS_TOKEN`，重活（MG5/Pythia8/Delphes）在云端容器里跑。
本地脚本	`scripts/run_*.py` 是**同一套逻辑**，直接在本机跑；已修复为尊重 `MG5_PATH`/`MA5_PATH`。**本工作区采用此模式**（云不可达）。
DeepSeek 运行时	只负责**编排**（调 `magnus` CLI、搬运文件），本身不做重计算；提示词里也明确要求遵守技能中的 MG5 状态机。

云不可达时的三条应对

1. 优先走 `scripts/` 本地入口（推荐，已验证端到端）；
2. 若只需要模型/宽度类结果，用 `src/skills/light-scalar-decay` 等纯本地技能，绕开 Magnus；
3. 让 agent **复用已有事例样本**，不要重跑重活——本机热受限时尤其重要，且重复产生事例既慢又可能触发过热关机。

6.3 机器散热与资源节流（本机重点）

本机实测温度行为

idle 即 67–71 °C（`x86_pkg_temp`），**单核满载可冲到 100 °C（crit）**，曾因 FeynRules 的 16 并行内核负载触发**过热关机**。watchdog 会在超温时直接杀掉 MG5/UFO 任务。

可用手段（均**无需 root**）：

手段	说明与建议
SIGSTOP/SIGCONT 温度门控	**首选**。循环监控：> 83°C 时 `kill -STOP`，< 62–65°C 时 `kill -CONT`；长任务自动暂停/恢复且不丢进度（参考 `darkhiggs/codexb_repro/run_gated.sh`）。
`cpulimit`	**本机已弃用**：会提前退出并把子进程变孤儿（任务显示完成但进程仍在跑、日志为空）。若必须用，配 `nice 19 + taskset -c 0`，并确认进程真在跑、日志在写。
`taskset` + `nice`	限核限优先级，安全但不如门控灵活：`nice -n 15 taskset -c 0,1 <cmd>`。
跑前降温	等核心温度 < 65°C 再起任务；监控文件 `/sys/class/thermal/thermal_zone8/temp`。
冷却门槛别设太低	低于 idle 基线（例如 < 52°C）会**永远等不到**。

手段	说明与建议
改 CPU 频率	`cpupower frequency-set -u 2000000` **需要 sudo**，无 root 时只能用上面的节流方案。

FeynRules `WriteUFO` 的安全跑法

`WriteUFO` 会拉起 16 个并行内核。安全做法：在 `.m` 文件里把 `FR$Parallel=False` 写在**`<< FeynRules` 之前**，再用 2 核限流运行：

```
cd /tmp && rm -rf ufo_test && mkdir ufo_test && cd ufo_test
nice -n 15 taskset -c 0,1 wolframscript -file ufo_test.m
```

解析类任务也要注意：numpy/matplotlib 分析虽轻，但 80×80 网格上每点做全量 ray–box 计算同样发热。优化办法：先把几何量（每质量的 $l_{\min}/l_{\max}/p$）**预计算一次**，耦合轴只做 exp 平均，可快约 80 倍。

`pkill -f` 会杀掉自己

`pkill -f <pattern>` 会匹配到它自己的命令行。用 `pkill -x <name>` 或按 PID kill。

6.4 运行目录与产物

```
outputs/                                  # python-agent 默认输出目录
+-- MyModel.fr                            # FeynRules 模型
+-- MyModel_UFO/                          # UFO 模型（particles/parameters/vertices.py）
+-- simulation.yaml                       # 仿真配置
+-- pp_XX/                                 # MadGraph 过程目录
    +-- Cards/param_card.dat, run_card.dat
    +-- Events/run_01/
        +-- unweighted_events.lhe.gz          # parton 级
        +-- tag_1_pythia8_events.hepmc.gz     # hadron 级
        +-- tag_1_delphes_events.lhco.gz      # reco 级
```

先存原始结果，再画图

把所有 $(m, \sin\theta) \to c\tau, A, \mathrm{BR}, N_{\mathrm{sig}}, \mathrm{excluded}$ 存成 **JSON**（如 `results_standard.json`），绘图与核对一律从 JSON 读，**不要在绘图脚本里重算物理量**。否则一旦公式有误，图和表会“错得一致”而无法察觉。

6.5 修改 agent 内部文件的正确姿势

1. 改**源仓库**（`src/agents/`、`src/skills/`、`scripts/`）；
2. 重跑 `./setup_collider_agent.sh`（`--force`）把新副本刷进 `deepseek-agent/`；
3. 若脚本本身用 `re.sub` 注入内嵌代码，注意替换串里的`\n` 会被转义——**要用 lambda 形式的替换**；

4. 重跑第 5 节的 L1 冒烟测试确认没改坏。

6.6 常见故障快速排查

症状	可能原因	处理
`ModuleNotFoundError: install_madgraph`	仓库缺文件（第 3.7 节）	确认本地修复仍在；重跑 `import agent, tools` 检查
`pytest` 报 collection error	同上（缺失模块）	修复后应为 38 passed
`unrecognized arguments: --timeout`	未装 `pytest-timeout`	去掉该参数或用 shell `timeout`
MG5 找不到 LHAPDF / `libLHAPDF.so`	`LD_LIBRARY_PATH` 未在进程启动前设置	`source local_env.sh` 或改用 `run_mg5.sh`
MG5 参数不生效（nevents 被忽略）	`set …` 写在第二个 `done` 之后	按技能里的状态机重写 `--commands`
MA5 启动崩溃（ReadTimeout）	更新检查器只捕 `ConnectionError`	已在 `system/checkup.py` 加入 `requests.exceptions.Timeout`
Magnus `connection refused`	云端未起	回退本地 `scripts/`（第 6.2 节）
任务“完成”但进程还在跑、日志为空	`cpulimit` 孤儿进程	停用 cpulimit，改温度门控
机器过热关机	16 并行 Wolfram 内核 / 满载 MG5	`FR$Parallel=False` + `taskset` 限核 + 温度门控
micrOMEGAs 链接失败 / `lib/serv.a` 损坏	CalcHEP 并行 make	改单线程 `make -j1`
pip 安装 403	清华镜像在本网络不可用	换阿里云等镜像
`$FeynRulesPath` 未定义	ColliderAgent 模板不自行设置	已打自定位补丁；确认补丁还在

6.7 安全与保密

- `python-agent/.env` 与 `deepseek-agent/.env` 内含**真实 API key / Magnus token**，权限应保持 `600`，且确认被 `.gitignore` 覆盖。
- 模型**看不到 API key**：key 只由 `agent.py` 用于自身向 API 认证，不会进入对话内容或工具输出。
- 提交代码前检查 `.env`、`e2e_test.log`、大规模事例样本是否被误加入版本库。
- 共享机器上跑长任务，注意磁盘（事例文件很大）与他人资源占用；建议用 `taskset` 限核而不是占满整机。

A 命令速查表

目的	命令
载入环境	`source ColliderAgent/local_env.sh`
安装 DeepSeek 代理	`./setup_collider_agent.sh [--force]`
导包冒烟	`python-agent/.venv/bin/python -c "import agent, tools, utils"`
全量测试	`cd python-agent && ./.venv/bin/python -m pytest tests/ -q`
测试收集检查	`./.venv/bin/python -m pytest tests/ --collect-only -q`
代理提示自检	`python -c "import agent; agent.build_system_prompt()"`
代理冒烟运行	`python agent.py "run pwd and report"`
从文件跑任务	`python agent.py --task-file prompt.md --workdir DIR`
FeynRules 校验	`python scripts/run_feynrules_validation.py --model M.fr --symbol S`
UFO 导出（节流）	`nice -n 15 taskset -c 0,1 wolframscript -file ufo.m`
MG5 编译	`python scripts/run_madgraph_compile.py --ufo UFO --process "p p > l+ l-"`
MG5 产生事例	`python scripts/run_madgraph_launch.py --process DIR --commands "…"`
MA5 分析	`python scripts/run_madanalysis_process.py --events E.lhe.gz --script S`
MG5 直接调用	`MadGraph5/MG5_aMC_v3_7_2/run_mg5.sh`
MA5 直接调用	`MadAnalysis5/madanalysis5-1.11.1/run_ma5.sh`
温度监控	`cat /sys/class/thermal/thermal_zone8/temp`

B 环境变量速查

变量	归属	本机取值（示例）
`MG5_PATH`	local_env / .env	…/MadGraph5/MG5_aMC_v3_7_2/bin
`MA5_PATH`	local_env / .env	…/MadAnalysis5/madanalysis5-1.11.1/bin
`DELPHES_PATH`	local_env / .env	…/ColliderSim/delphes

变量	归属	本机取值（示例）
DELPHES_CARD_CMS/ATLAS	.env	delphes/cards/delphes_card_*.tcl
MICROMEGAS_ROOT	local_env / .env	…/ColliderSim/micromegas_6.3.0
LHAPDF_LIB / LHAPDF_BIN	local_env	MG5 内 HEPTools/lhapdf6_py3/{lib,bin}
WOLFRAM_KERNEL_PATH	.env	/usr/local/Wolfram/Wolfram/14.3/Executables/WolframKernel
WOLFRAMSCRIPT_PATH	.env	/usr/bin/wolframscript
MAGNUS_ADDRESS	.env	http://162.105.131.134:3011（本机不可达）
MAGNUS_TOKEN	.env	云端 token（**保密**）
COLLIDER_AGENT_MODEL	.env	openai/gpt-5-mini
OPENAI_API_KEY / OPENAI_BASE_URL	.env	LLM 提供方配置（**保密**）
DEEPSEEK_API_KEY / DEEPSEEK_MODEL	deepseek-agent/.env	key + deepseek-v4-flash
DEFAULT_NEVENTS	.env	10000

C 相关文件与文档索引

路径	内容
ColliderAgent/README.md / README_zh.md	上游总说明（架构、技能、示例提示词）
ColliderAgent/DEPENDENCIES.md	本机依赖完成情况、代码修复、热节流记录
ColliderAgent/python-agent/README.md	ADK 智能体安装、工具参考、项目结构
ColliderAgent/scripts/README.md	Magnus 蓝图与脚本参数说明
ColliderAgent/deepseek-agent/README.md	DeepSeek 运行时说明与用法
ColliderAgent/local_env.sh	环境变量一键载入（本工作区新增）
setup_collider_agent.sh	DeepSeek 代理一键安装（内嵌全部文件）
docs/INSTALL_NOTES.md	MG5 / MA5 / LHAPDF 安装记录与补丁
docs/SIMULATION_NOTES.md	物理/数据/热机/绘图/Agent 各类注意事项
docs/SESSION_HANDOFF.md	项目状态交接（新会话先读）
docs/README.md	文档索引
本文	docs/ColliderAgent_install_test_run_notes.pdf

D 参考链接

- ColliderAgent 官方仓库：https://github.com/HET-AGI/ColliderAgent
- Magnus 云平台：https://github.com/Rise-AGI/magnus
- MadGraph5_aMC@NLO：https://launchpad.net/mg5amcnlo
- MadAnalysis5：https://github.com/MadAnalysis/madanalysis5
- Delphes：https://github.com/delphes/delphes
- Pythia8：https://pythia.org/
- FeynRules：https://feynrules.irmp.ucl.ac.be/
- Wolfram Engine 免费 license：https://wolfram.com/engine/free-license
- LHAPDF：https://lhapdf.hepforge.org/
- micrOMEGAs：https://lapth.cnrs.fr/micromegas/

本文档基于 ColliderSim 工作区实机验证结果编写
实测命令、版本号与测试计数均来自本机复跑